Grafos

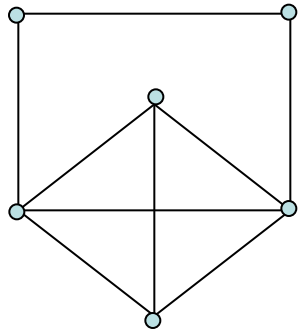
IFRN

Problemas Algorítmicos

- Categorias de Problemas Algorítmicos
  - Problema de Decisão
  - Problema de Localização
  - Problema de Otimização

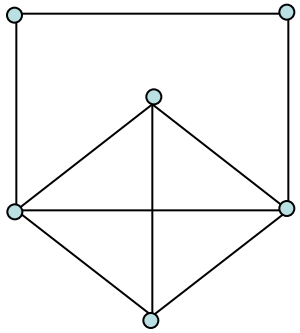
Problemas Algorítmicos

- Categorias de Problemas Algorítmicos
  - Problema de Decisão
    - Consiste em responder Sim ou Não à determinada pergunta
    - Ex.: No grafo abaixo existe uma clique >= 3?



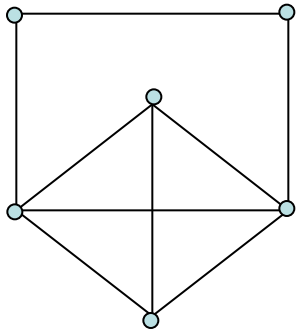
Problemas Algorítmicos

- Categorias de Problemas Algorítmicos
  - Problema de Localização
    - Consiste em encontrar, caso exista, determinada estrutura satisfazendo requisitos especificados por uma questão
    - Ex.: encontrar uma clique >= 3?



Problemas Algorítmicos

- Categorias de Problemas Algorítmicos
  - Problema de Otimização
    - Consiste em encontrar determinada estrutura satisfazendo um critério de otimização pré-definido
    - Ex.: encontrar uma clique de tamanho máximo no grafo abaixo



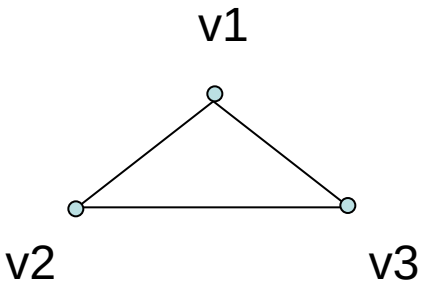
Conexidade e Distância

- Aplicações:
  - Telefonia
  - Tráfego
  - Redes de comunicação
  - Indústria Eletrônica – caminhos CI
  - Indústria de Confecções – Otimização de corte tecido

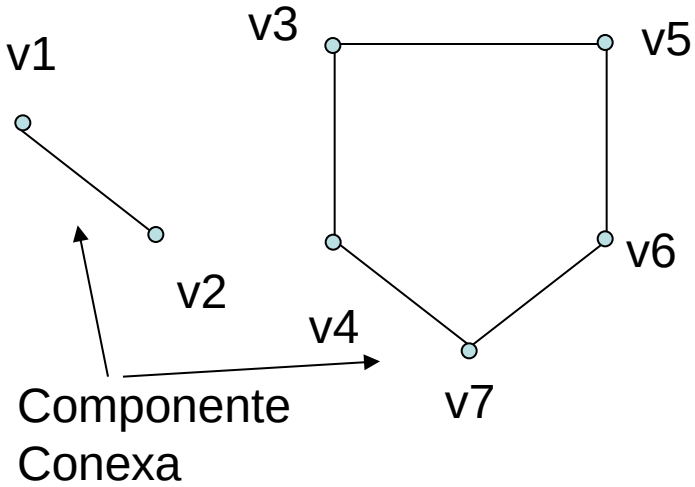
Conexidade e Distância

- Grafo Conexo
  - Um grafo G(V,A) é dito ser conexo ou simplesmente conexo ou S-Conexo. Se há pelo menos uma seqüência de arestas ligando cada par de vértices do grafo G.

Conexo



Não-Conexo

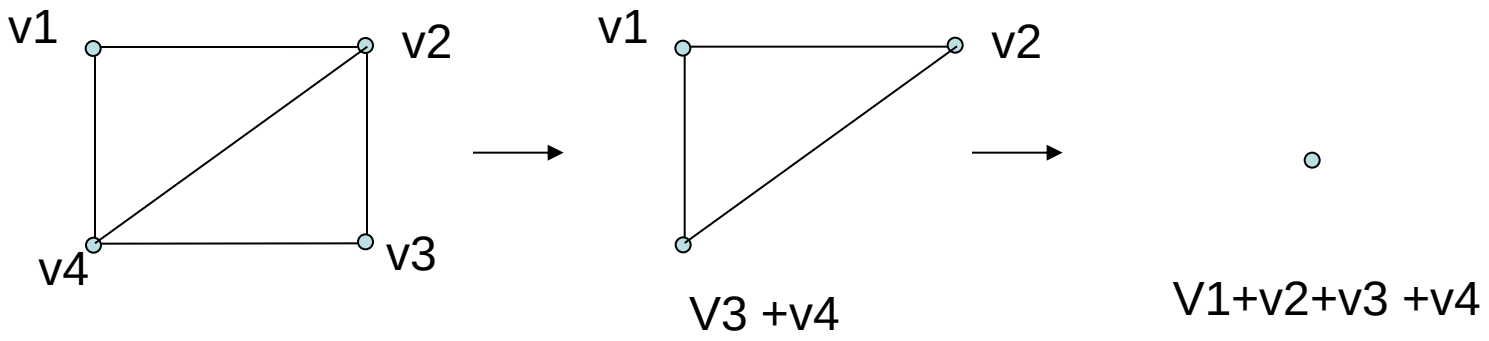


Conexidade e Distância

- Grafo Conexo
  - Um grafo NÃO conexo consiste de dois ou mais subgrafos conexos (componentes)

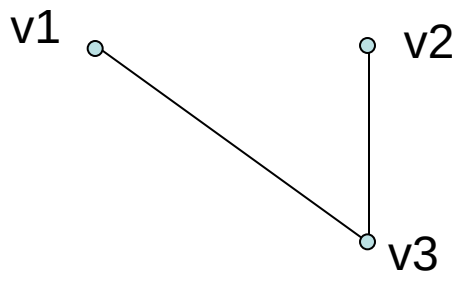
Conexidade e Distância

- Algoritmo para Conexidade
  - Reduzir cada componente do grafo a um único vértice. Processo de redução sequêncial onde todos os vértices adjacentes a um dado vértice são fundidos com ele



Conexidade e Distância

- Algoritmo para Conexidade
  - OBS.: qualquer grafo simples com n vértices e mais que (n-1)(n-2)/2 arestas é conexo



N=3
Num_arestas > (3-1)(3-2)/2=1

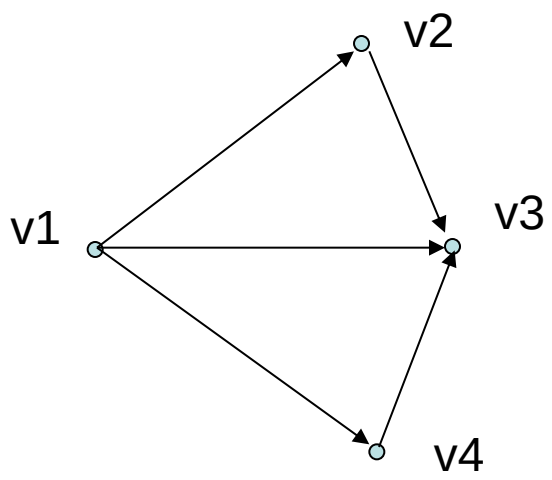
Conexidade e Distância

- Algoritmo para Conexidade de Goodman

```
P0 [inicialização]
H=Vg; // todos os vértices
c=0;
P1[gere a próxima componente conexa]
Enquanto (H!=vazio){
   selecione um vértice v0 pertencente a H
   Enquanto (v0 for adjacente a algum vértice v pertencente a H){
     H=grafo resultante da fusão de v com v0;
    v0=fusão de v com v0;
   }
   remova v0, isto é, faça     H=H-v0;
                               c=c+1;
}
P2[Testa o tipo de conexidade]
Se(c>1)
   G é não conexo
senao
   G é conexo
```

Conexidade e Distância

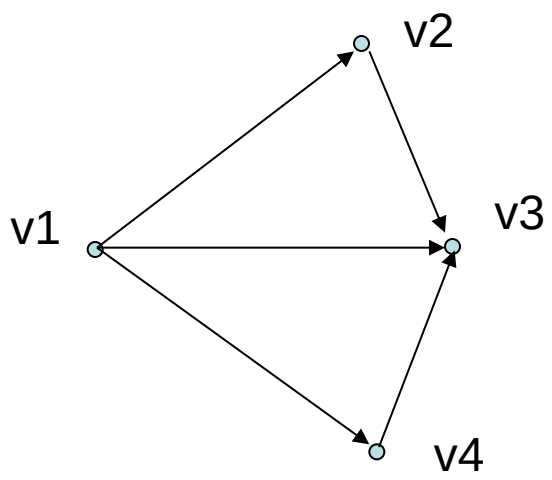
- Antecessor de um vértice
  - É todo Vj que seja extremidade inicial de uma aresta que termina em Vi. Sua notação é $\Gamma^{-}(v_i)$



Antecessores de v3= {v1,v2,v4}

Conexidade e Distância

- Sucessor de um vértice
  - É todo Vj que seja extremidade final de uma aresta que termina em Vi. Sua notação é $\Gamma^{+}(v_i)$



Sucessores de v1= {v2,v3,v4}

Conexidade e Distância

- Fechos Transitivos Diretos
  - Um fecho transitivo direto de um vértice Vi é o conjunto de todos os vértices que podem ser atingidos a partir de Vi, em um número qualquer de etapas. Sua notação é $\hat{\Gamma}^{+}(v_i)$

$$\hat{\Gamma}^{+}(v_i) = \{v_i\} \cup \left[\bigcup_{k=1}^{n} \hat{\Gamma}^{+k}(v_i) \right]$$

$$\Gamma^{+}(v_i) = \Gamma^{+}(v_i)$$

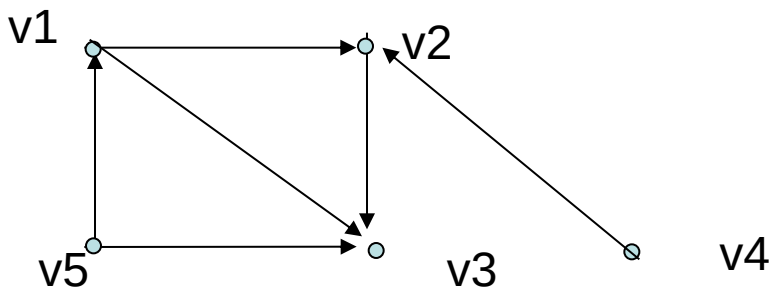
$$\Gamma^{+2}(v_i) = \Gamma^{+}(\Gamma^{+}(v_i))$$

$$\ldots$$

$$\Gamma^{+n}(v_i) = \Gamma^{+}(\Gamma^{+(n-1)}(v_i))$$

Conexidade e Distância

- Fechos Transitivos Diretos



$$\hat{\Gamma}^{+}(v_1) = \{v_1, v_2, v_3\}$$

$$\hat{\Gamma}^{+}(v_5) = \{v_5, v_1, v_3, v_2\}$$

Conexidade e Distância

- Fechos Transitivos Inverso
  - Um fecho transitivo inverso de um vértice Vi é o conjunto de todos os vértices que podem atingir Vi, em um número qualquer de etapas. Sua notação é

$$\hat{\Gamma}^{-}(v_i) = \{v_i\} \cup \left[\bigcup_{k=1}^{n} \Gamma^{-k}(v_i) \right]$$

$$\Gamma^{-}(v_i) = \Gamma^{-}(v_i)$$

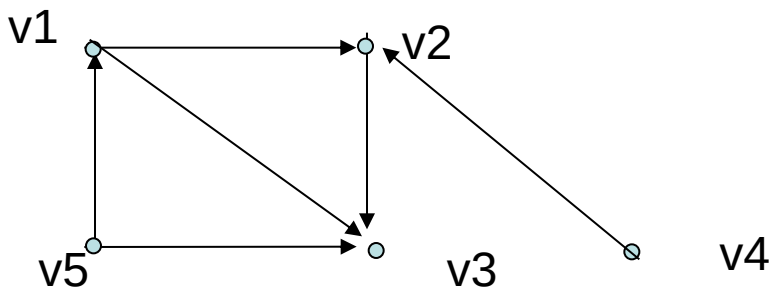
$$\Gamma^{-2}(v_i) = \Gamma^{-}(\Gamma^{-}(v_i))$$

...

$$\Gamma^{-n}(v_i) = \Gamma^{-}(\Gamma^{-(n-1)}(v_i))$$

Conexidade e Distância

- Fechos Transitivos Inverso

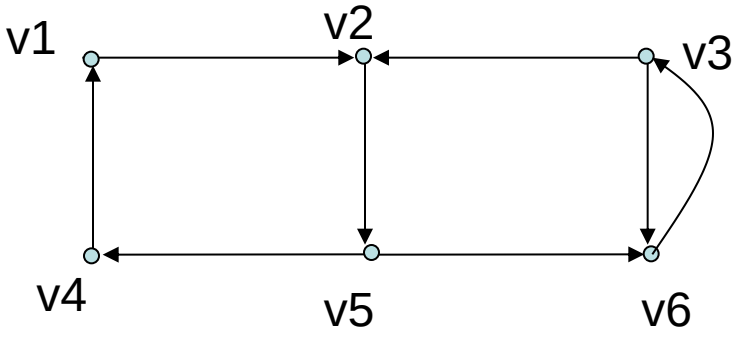


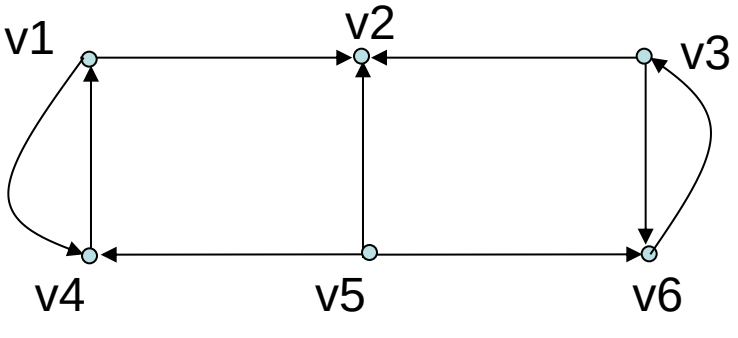
$$\hat{\Gamma}^{-}(v_1) = \{v_5, v_1\}$$

$$\hat{\Gamma}^{-}(v_5) = \{v_5\}$$

Conexidade e Distância

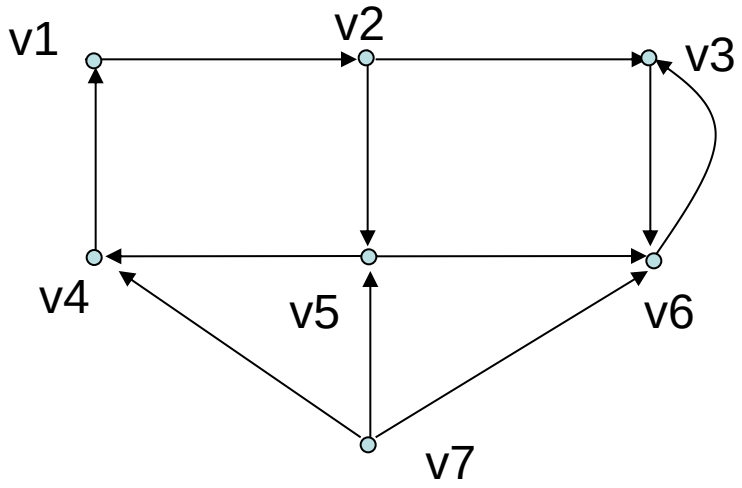
- Grafo Fortemente Conexo
  - No caso de grafos orientados, um grafo é dito ser fortemente conexo (F_Conexo) se todo par de vértices participa de um circuito, ou seja, se cada par de vértices participa de um circuito. Isto significa que cada vértice pode ser alcançável partindo-se de qualquer outro vértice de grafo.





Conexidade e Distância

- Componente Fortemente Conexa
  - Um grafo G(V,A) que não é fortemente conexo é formado por pelo menos dois subgrafos F_Conexos (fortemente conexo). Cada um desses grafos é dito ser uma componente fortemente conexa de G.



Conexidade e Distância

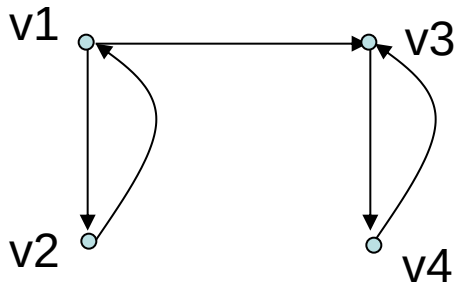
- Método para achar as componentes F-Conexas de um Grafo

```
H = V;
Enquanto(H!=vazio){
  Ache a componente conexa em relação a Vi
```

$$y(v_i) = \hat{\Gamma}^{+}(v_i) \bigcap \hat{\Gamma}^{-}(v_i)$$

$$H = H - y(v_i)$$

```
}
```



Conexidade e Distância

- Método para achar as componentes F-Conexas de um Grafo usando a matriz R e Q

Matriz $R = \left\lfloor \Gamma_{ij} \right\rfloor$

$$\Gamma_{ij} = \begin{cases} 1, se\ o\ vértice\ v_j\ \ pode\ ser\ atingido\ \ partindo\ no\ vértice\ v_i \\ 0, caso\ contrário \end{cases}$$

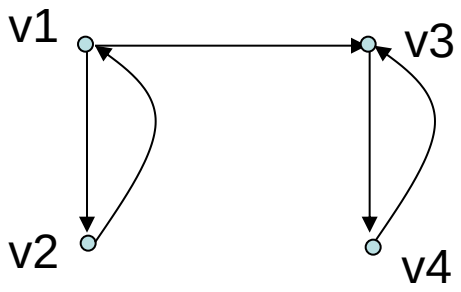
$$Q = R^t$$

Para se encontrar as componentes fortemente conexas faz-se

$$R \otimes Q$$

Conexidade e Distância

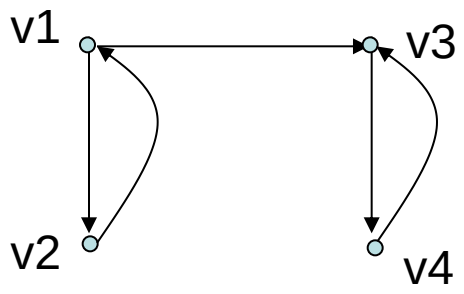
- Ex.:

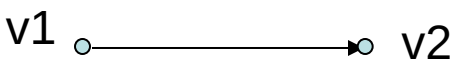


$$R = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \quad Q = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \quad R \otimes Q = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Conexidade e Distância

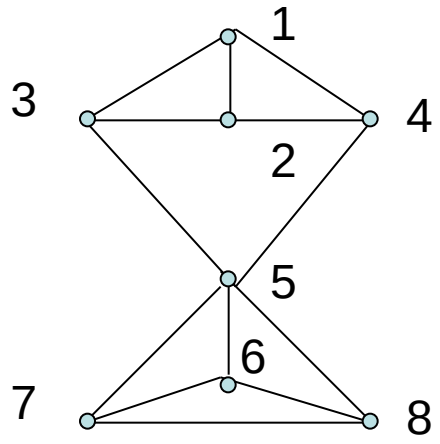
- Grafo Reduzido
  - Cada vértice corresponde a uma componente fortemente conexa do grafo. Sua notação é $G^*(V^*, A^*)$





Conexidade e Distância

- Vértice de Corte
  - Um vértice é dito ser um vértice de corte se sua remoção(juntamente com as arestas a ele conectadas) provoca uma redução na conexidade do grafo

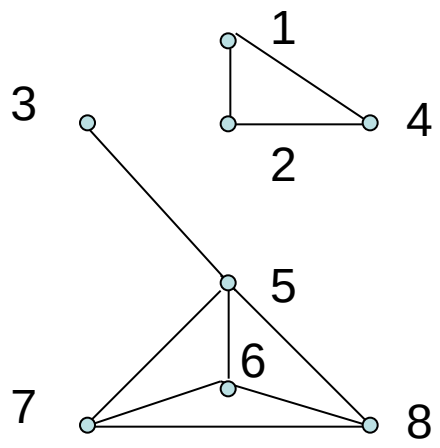


Ex.: {3,4}
{5} -> vértice de corte de cardinalidade mínima

  - A conectividade do grafo é 1(pois removendo 5 desconectamos o grafo)

Conexidade e Distância

- Arestas de Corte
  - Uma aresta é dita ser uma aresta de corte se sua remoção provoca uma redução na conexidade do grafo

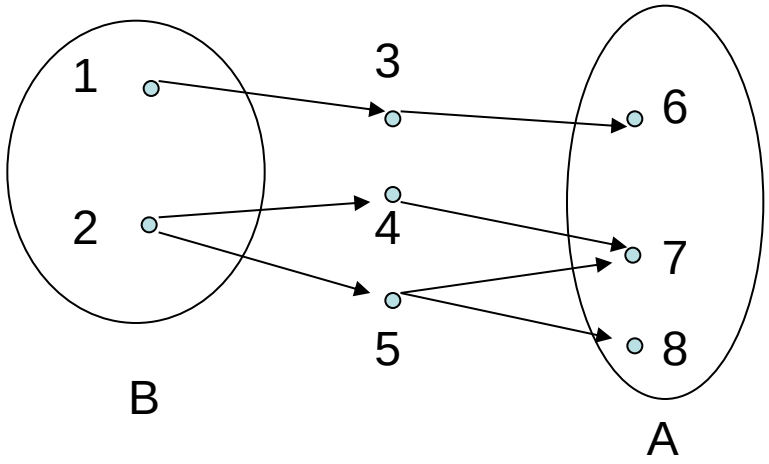


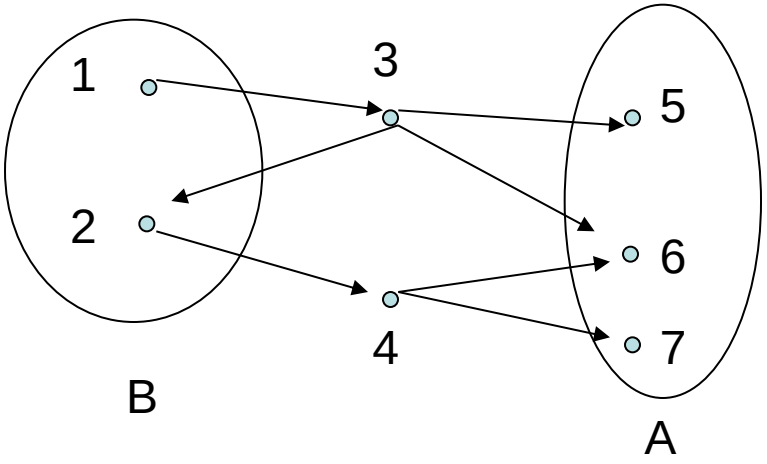
Ex.: {(1,3),(2,3),(4,5)}
{(3,5,(4,5)} -> arestas de corte de cardinalidade mínima

  - A conectividade de arestas do grafo é 2(pois removendo (3,5) e (4,5) desconectamos o grafo)

Conexidade e Distância

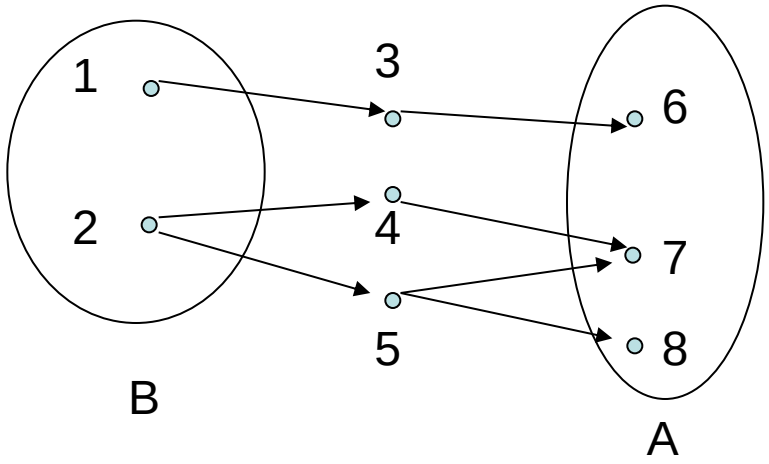
- Base
  - Uma base de um grafo G(V,A) é um subconjunto $B \subset V$, tal que:
    - Dois vértices quaisquer de B não são ligados por nenhum caminho
    - Todo vértice não pertencente a B pode ser atingido por um caminho partindo de B

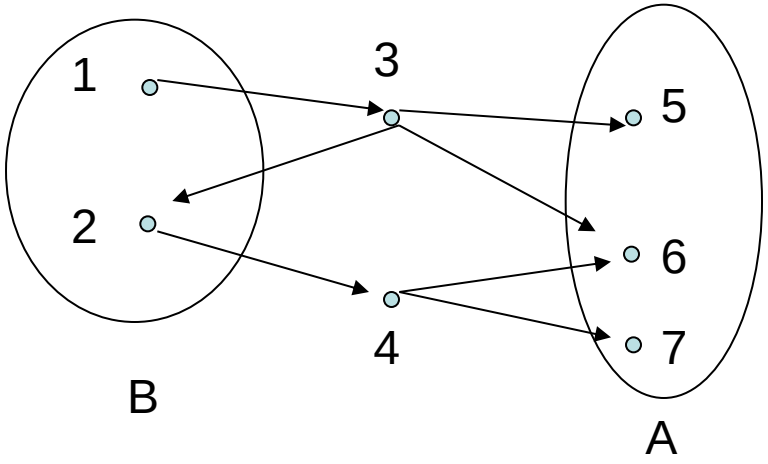




Conexidade e Distância

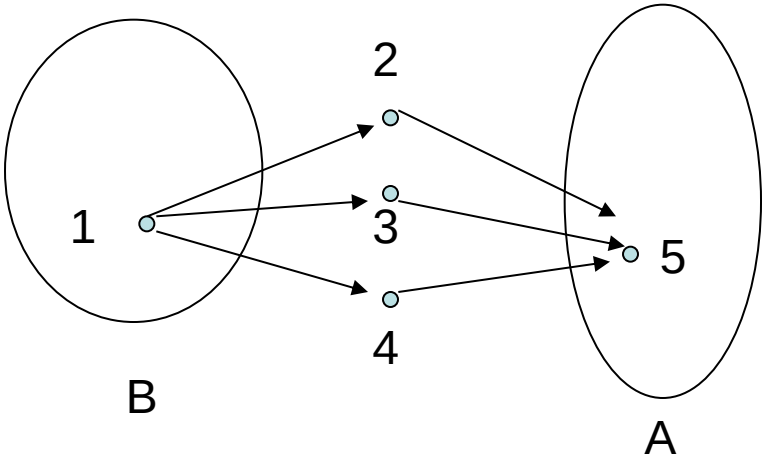
- Anti-Base
  - Uma anti-base de um grafo G(V,A) é um subconjunto $B \subset V$, tal que:
    - Dois vértices quaisquer de A não são ligados por nenhum caminho
    - Todo vértice não pertencente a A pode atingir A por um caminho





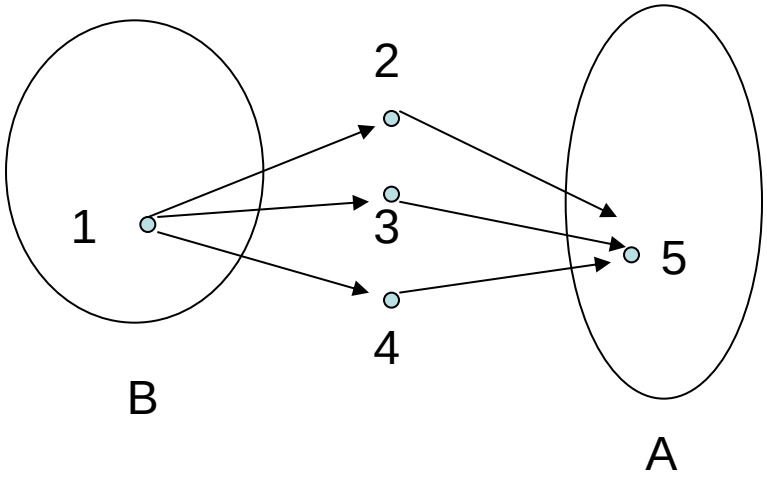
Conexidade e Distância

- Raiz
  - Se a base de um grafo G(V,A) é um conjunto unitário então esta base é a raiz de G



Conexidade e Distância

- Anti-Raiz
  - Se a anti-base de um grafo G(V,A) é um conjunto unitário então esta anti-base é a anti-raiz de G



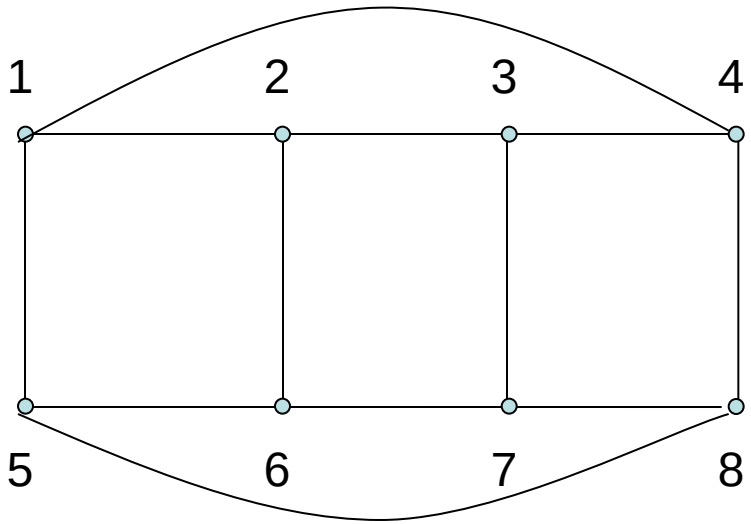
Conexidade e Distância

- Distância
  - Dado dois vértices v e w pertencentes ao grafo G(V,E), denomina-se distância entre v e w o comprimento do menor caminho entre esses dois vértices. Se não houver caminho, a distância é infinita
  - Notação d(v,w)
  - Propriedades:
    - d(w,v)>=0 ou d(w,v)=0 p/ w=v
    - d(w,v)=d(v,w) apenas p/ grafos não direcionados
    - d(z,v) + d(v,w) >= d(z,w)

Conexidade e Distância

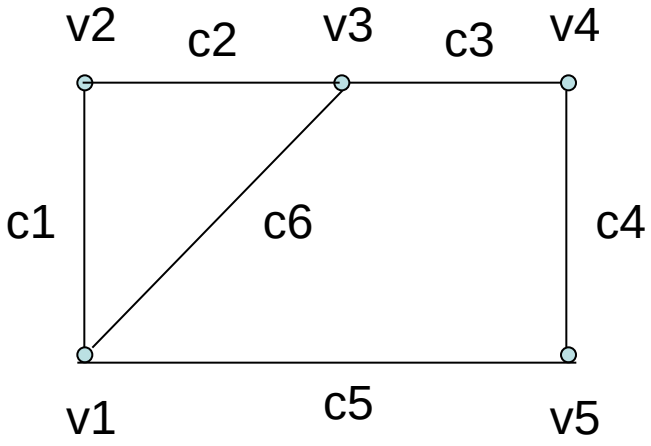
- Distância

d(1,8)? d(1,4)? d(1,3)?



Caminhos

- É uma seqüência de arestas onde o vértice final de uma aresta é o vértice inicial da próxima



{c1,c2,c4,c5,c6} {c2,c3,c4,c5}

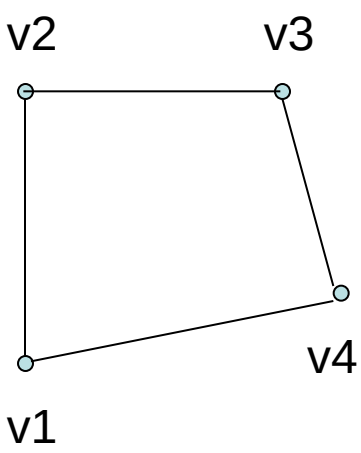
{v1,v2,v3,v4,v5} {v2,v3,v1,v5,v4}

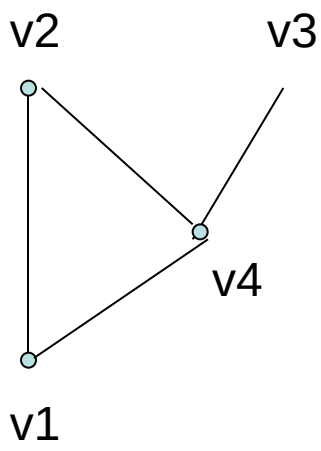
Caminhos

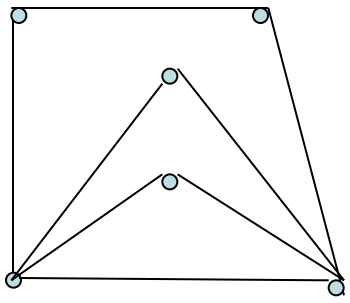
- Um caminho de k vértices tem (k-1) arestas, v1v2,v2v3,...,vk-1,vk e o comprimento do caminho é (k-1)
  - Caminho aberto(simples ou elementar): Todo os vértices são distintos. Ex.:{c2,c3,c4}
  - Caminho fechado ou ciclo: é aquele em que o vértice inicial é igual ao final, ou seja: v1,v2,...,vk,vk+1, onde vk+1=v1. Ex {c1,c2,c3,c4,c5} não é um caminho aberto

Caminhos

- Grafos Eulerianos (grafos de Euler)
  - São grafos onde é possível achar um caminho fechado (ciclo), passando em cada aresta uma única vez
  - Quais são os grafos de Euler?

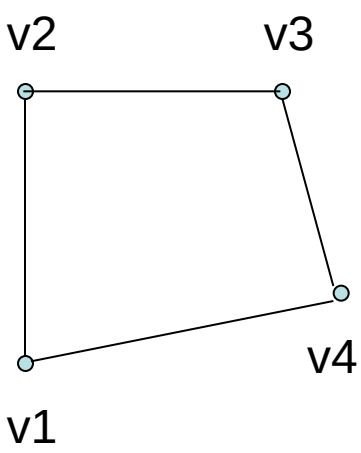


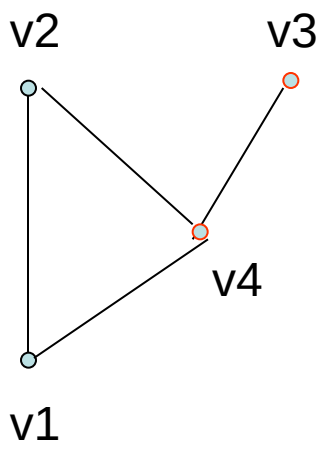


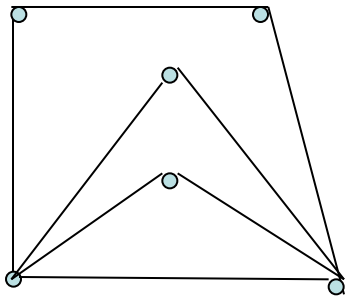


Caminhos

- Grafos Eulerianos (grafos de Euler)
  - Teorema: um grafo conexo G é um grafo de Euler se e somente se todos os seus vértices são grau par.

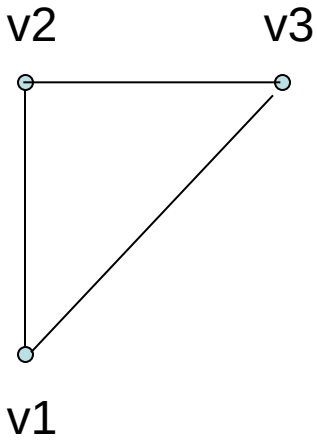


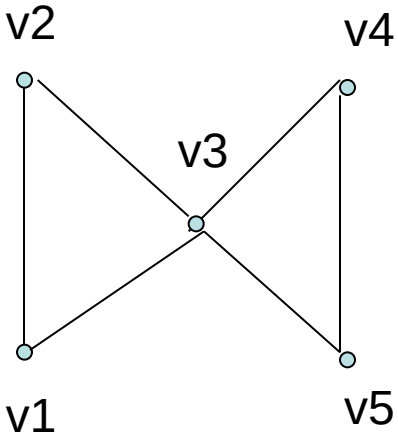




Caminhos

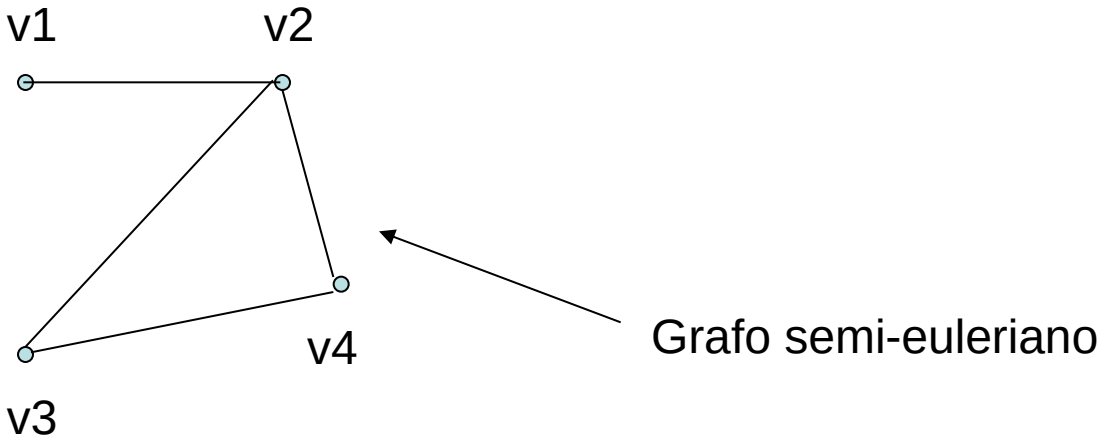
- Caminhos Eulerianos
  - São caminhos que passam por cada aresta um vez, passando por todas
  - Teorema: baseado no grau dos vértices do grafo: existe um caminho Euleriano em um grafo se:
    - Não ocorrer vértices de grau ímpar (inicia e termina no mesmo vértice)





Caminhos

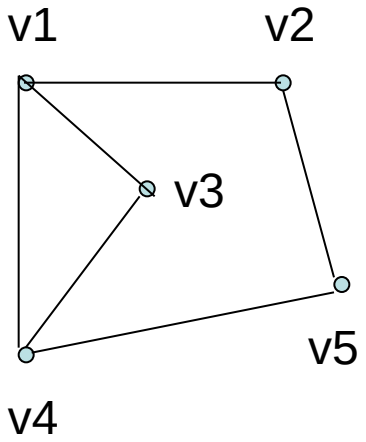
- Caminhos Eulerianos
  - Teorema: baseado no grau dos vértices do grafo:existe um caminho Euleriano em um grafo se:
    - Existem 2 vértices de grau ímpar (inicia em um vértice ímpar e termina em outro vértice ímpar)



Caminhos

- Algoritmo para verificar a existência de um caminho euleriano

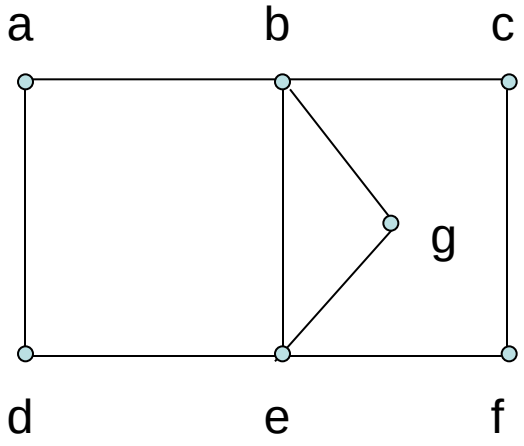
```
grau=0;   soma=0;   matadj[][];   N=numero de linhas da matriz;   f=0;//linha atual;
Enquanto(soma<=2)e(f<=N) {
          grau=0;
          para(g=0;g<N;g++){
                    grau+=matadj[f][g];
          }
          se grau mod 2 == 1  // ímpar
                    soma++
          f++;
}
Se (soma>2)         NÃO EXISTE CAMINHO
Senao               EXISTE CAMINHO
```



	1	2	3	4	5	*grau*
1	0	1	0	1	0	
2	1	0	0	1	1	
3	1	0	0	1	0	
4	0	1	1	0	1	
5	0	1	0	1	0	

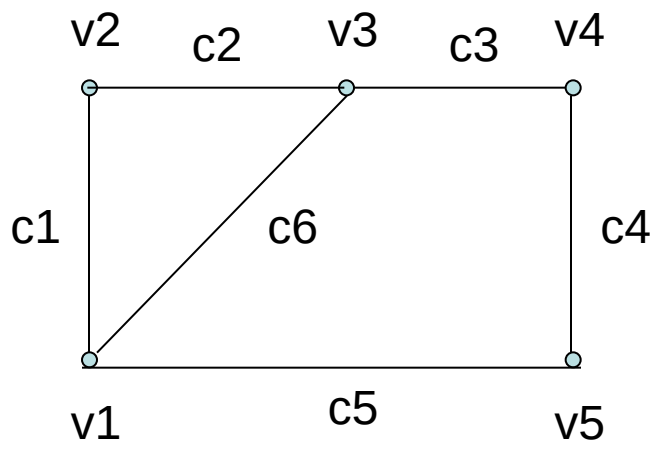
Caminhos

- Método de Fleury para traçar um ciclo euleriano
  - Inicie em qualquer vértice v e atravesse as arestas de uma maneira arbitrária, seguindo as seguintes regras
  - R1) apague a aresta que foi visitada e se algum vértice ficar isolado, apague-o também.
  - R2) em cada estágio, nunca atravesse uma aresta, se naquele momento a sua remoção divide o grafo em duas ou mais componentes não triviais (excluindo os vértices isolados).



Caminhos

- Ciclos e Caminhos Hamiltonianos
  - Um ciclo hamiltoniano em um grafo conexo G é definido com um caminho simples fechado em que cada vértice de G é visitado uma única vez (com exceção do nó inicial == final)
  - Um ciclo hamiltoniano em um grafo de n vértices tem n arestas



-Não é um grafo de euler

-Possui ciclo hamiltoniano?

Caminhos

- Condições para existência de um caminho Euleriano em um grafo
  - Grafo de euler ou semi-euler
- Condições para existência de um caminho Hamiltoniano em um grafo
  - Não tem solução

Problema do Carteiro Chinês

- A primeira referência ao problema foi em 1962 em uma revista chinesa, tendo a designação ficada associada ao problema
- Pode ser formulado como o problema de encontrar um ciclo Euleriano de menor custo
  - Exemplo: Considere um bairro de uma cidade em que um carteiro é responsável por distribuir correspondências. Neste modelo as arestas ponderadas representaram as ruas e suas respectivas distâncias e os vértices os cruzamentos. Para que a correspondência seja entregue, é necessário que todas as ruas sejam percorridas pelo menos uma vez e que ao final o carteiro volte a estação do correio. O problema consiste em determinar o menor caminho possível para que o carteiro faça seu trabalho.
  - Problemas similares: recolhimento do lixo, inspeção de linhas de transmissão, inspeção de linhas telefônicas, etc.

Problema do Carteiro Chinês

- Encontrar o menor caminho(sendo de euler)
  - Problema do carteiro chinês: início ao fim passando em todas as arestas uma vez (menor caminho) – caminho de euler
    - Determinar o custo do menor caminho
    - Solução 1: se o grafo for euleriano, aplicar algoritmo de Fleury
    - Solução 2: se o grafo não for de euler aplicar o algoritmo de carteiro chinês (Christofides)

Algoritmo do carteiro chinês

P1: determinar os vértices de grau ímpar

P2: construa a matriz de distancia D, com os vértices de grau ímpar (Dijkstra)

P3: determinar o par de vértices c/ menor caminho através da matriz D

P4: construa um caminho artificial de vi para vj com custo encontrado em P3

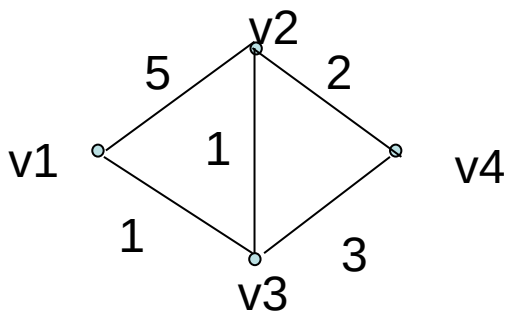
P5: elimine de D as colunas e linhas correspondentes vi e vj

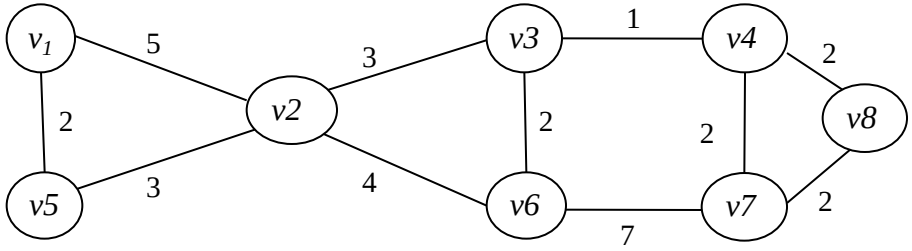
P6: se houver linhas e colunas em D, volte para P3

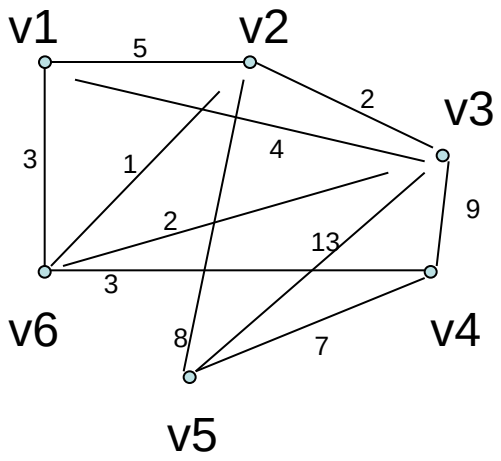
P7: ache o caminho de euler (fleury)

P8: o custo será igual à soma dos custos de todas as arestas acrescida dos custos das arestas encontradas em P3

Algoritmo do carteiro chinês







Problema do Menor Caminho

- Algoritmo Dijkstra
  - Edsger Dijkstra, ganhador do prêmio Turing em 1972
  - A idéia principal é o uso de um método guloso para o problema do caminho mínimo com origem única.
  - Funciona em grafo dirigido ou não dirigido com arestas de peso não negativo, em tempo computacional O([m+n]log n) onde m é o número de arestas e n é o número de vértices

Problema do Menor Caminho

- Algoritmo Dijkstra -determinar:
  - Seja G(V,A) e uma função L(V,W) pertencente aos reais e um vértice fixo V0 de V (fonte)
  - Problema: determinar v0,v1,...,vk, tal que $\sum_{i=0}^{k-1} l(v_i, v_{i+1})$ seja mínimo

$$L(v_i, v_j) = \begin{cases} \infty \; se \; não \; existir \;\; aresta(v_i, v_j) \\ 0 \quad v_i = v_j \\ custo \; se \; exitir \;\; aresta(v_i, v_j) \end{cases}$$

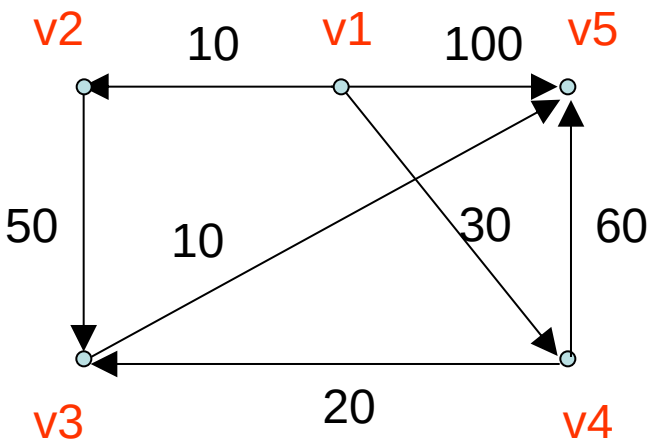
Problema do Menor Caminho Entre Dois Vértices

- Algoritmo Dijkstra -determinar:

```
S={v0};     D[v0]=0;
para cada v pertencente a V-{v0}
     D[v]=L(v0,V);
enquanto (s <> V){
     escolha o vértice w pertence V-S tal que D[w] seja mínimo
                    e congele o antecessor
     coloque w em S, isto é, faça S=S U {w}
     para cada v pertencente V-S faca{
            D[v]=min(D[v],d[w]+L(w,v))
            se(d[w]+L(w,v)<D[v])
                    D[v].antecessor=w;
     }
}
```

Problema do Menor Caminho Entre Dois Vértices

- Ex Dijkstra

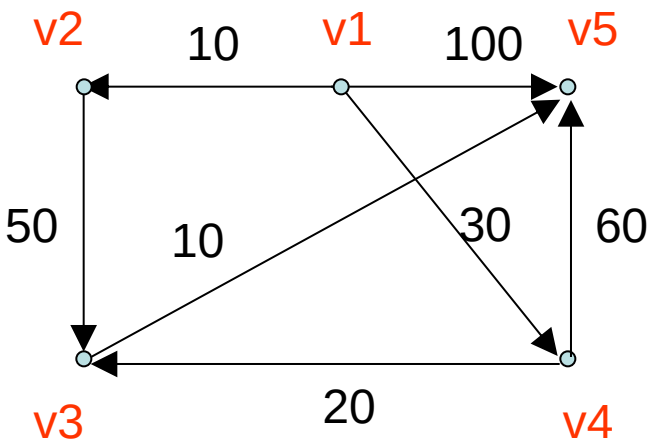


$$\begin{bmatrix} 0 & 10 & \infty & 30 & 100 \\ \infty & 0 & 50 & \infty & \infty \\ \infty & \infty & 0 & \infty & 10 \\ \infty & \infty & 20 & 0 & 60 \\ \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

Iteração	S	W	d(w)	d[2]	d[3]	d[4]	d[5]

Problema do Menor Caminho Entre Dois Vértices

- Ex Dijkstra

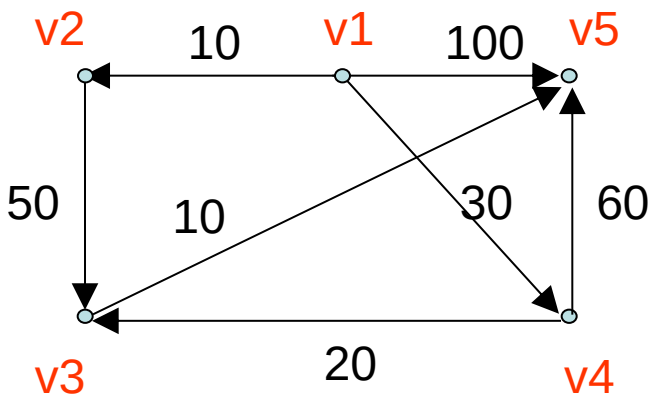


$$\begin{bmatrix} 0 & 10 & \infty & 30 & 100 \\ \infty & 0 & 50 & \infty & \infty \\ \infty & \infty & 0 & \infty & 10 \\ \infty & \infty & 20 & 0 & 60 \\ \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

Iteração	S	W	d(w)	d[2]	d[3]	d[4]	d[5]

Problema do Menor Caminho Entre Dois Vértices

- Ex Dijkstra

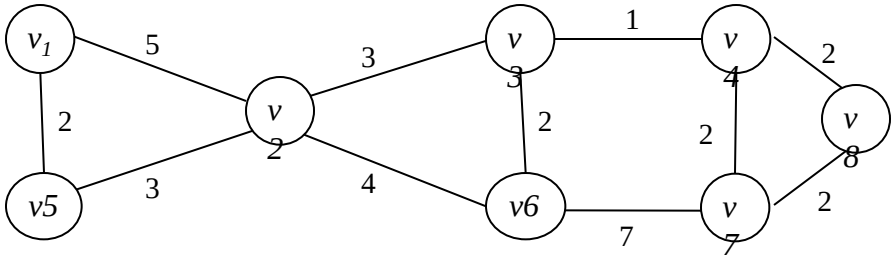


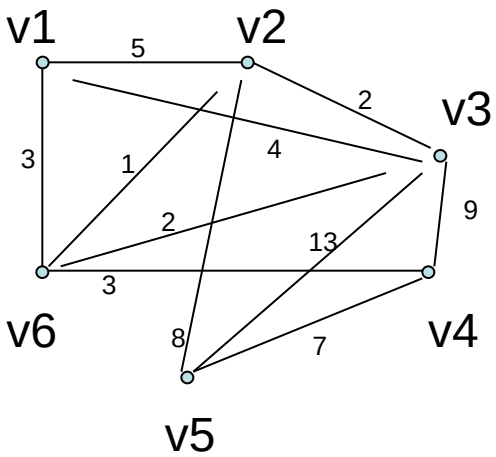
$$\begin{bmatrix} 0 & 10 & \infty & 30 & 100 \\ \infty & 0 & 50 & \infty & \infty \\ \infty & \infty & 0 & \infty & 10 \\ \infty & \infty & 20 & 0 & 60 \\ \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

Iteração	S	W	d(w)	d[2]	d[3]	d[4]	d[5]
Inicio	v1	-	-	10(v1)	∞	30(v1)	100(v1)
1	V1,v2	v2	10	-	∞,10+50 **60(v2)**	30,10+∞ **30(v1)**	100,10+ ∞ **100(v1)**
2	V1,v2,v4	v4	30	-	60,30+20 **50(v4).**limp	-	100,30+60 **90(v4)**
3	V1,v2,v4,v3	v3	50	-	-	-	90,50+10 **60(v3)**
4	V1,v2,v4,v3,v5	v5	60	-	-	-	-

Problema do Menor Caminho Entre Dois Vértices

- Exercício Dijkstra





Problema do Caixeiro Viajante

- Consiste em determinar o menor caminho, passando por todos os vértices uma única vez e retornando ao vértice de origem
- Métodos:
  - **Tentativa e erro** - A solução mais direta (simples em implementação) consiste em tentar todas as permutações possíveis, de modo a verificar por força bruta qual é o caminho de menor custo. Dado que a quantidade de permutações é o número de cidades, tal solução logo torna-se impraticável, não sendo utilizada.
  - Suponha um computador capaz de fazer 1 bilhão de adições por segundo. No caso de 20 cidades, o computador precisa apenas de 19 adições para dizer qual o comprimento de uma rota e então será capaz de calcular 1bilhão / 19 = 53 milhões de rotas por segundo. Para acharmos o tempo de execução total das diversas possibilidades teremos que realizar a seguinte conta:
  - Tempo = 19!/53 milhões = 2.3×10^9 seg
  - Ou seja, 73 anos!!

Problema do Caixeiro Viajante

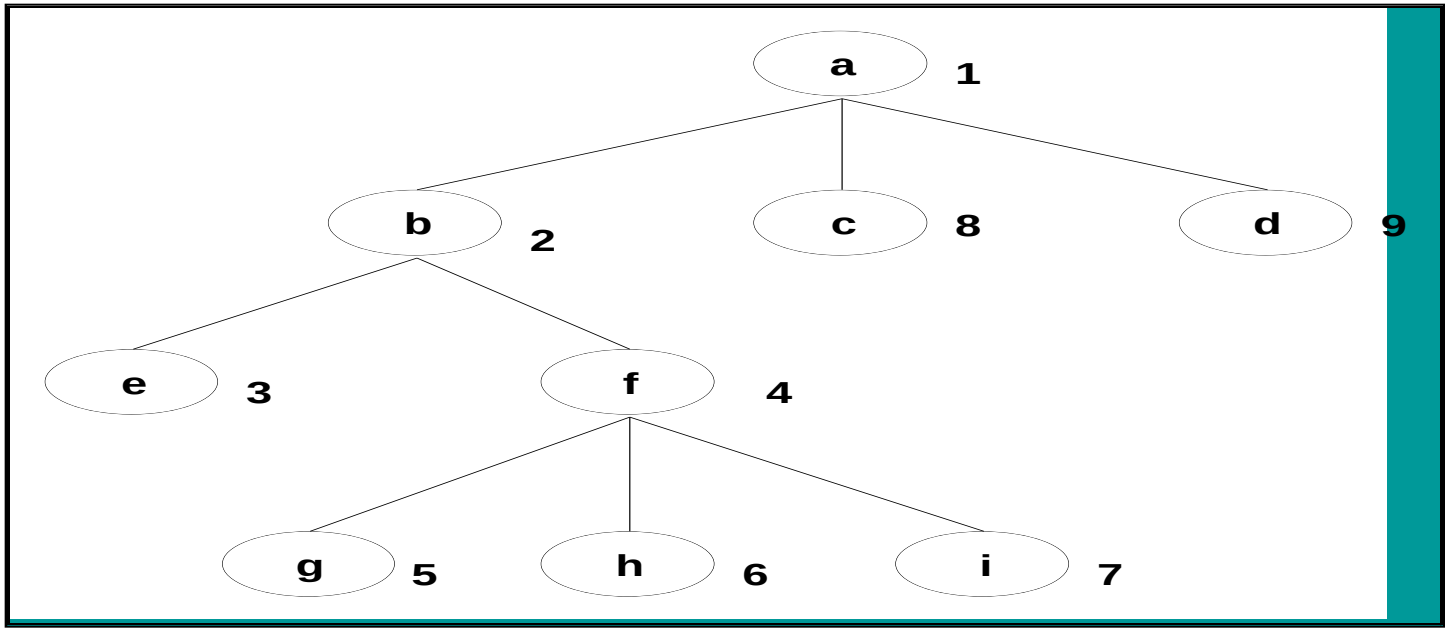
- Consiste em determinar o menor caminho, passando por todos os vértices uma única vez e retornando ao vértice de origem
- Métodos:
  - **Heurísticos** – não são a melhor solução, porém conseguem achar uma boa solução em um tempo viável

Problema do Caixeiro Viajante

- Consiste em determinar o menor caminho, passando por todos os vértices uma única vez e retornando ao vértice de origem
- Algoritmo de Roberts e Flores
  - Partindo de um vértice inicial, determinar um caminho, se o mesmo existir, que leva até o próprio vértice, e então, através do “backtracking” continuado, tenta determinar os demais

Problema do Caixeiro Viajante

- Backtracking
  - Ordem de visita aos nodos da árvore



a, b, e, (b), f, g, (f), h, (f), i, (f), (b), (a), c, (a), d

onde o caminho em backtracking é representado entre parênteses

Problema do Caixeiro Viajante

P1: Escolha um vértice inicial (xi)

P2: faça S={xi}

P3: adicione a S o primeiro vértice viável (xj não pertencente a S)

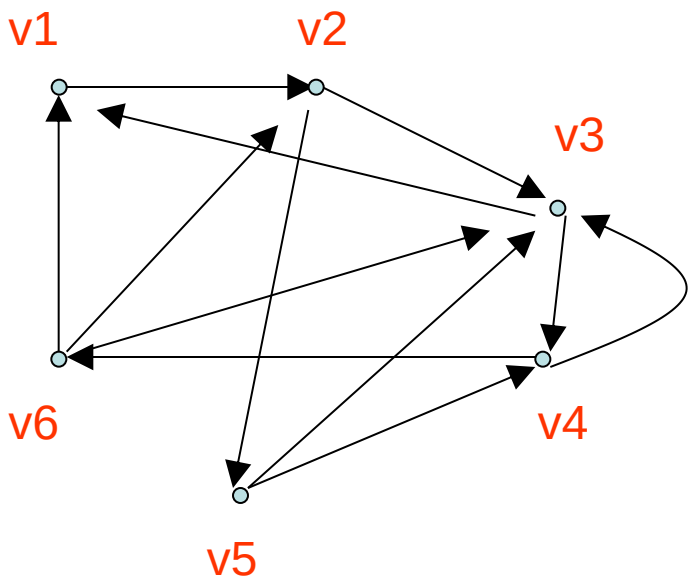
P4: repita o passo P3 enquanto houver vértices viáveis destino do último vértice viável (não pertencente S) encontrado na lista de adjacências

P5: Se S contém os n vértices de G, então a seqüência encontrada em S é um caminho hamiltoniano, digamos {xi,xj,...,xr}. Se existe uma aresta (xr,xi), então existe um ciclo hamiltoniano. Em caso contrário, não existe ciclo e deve-se fazer um "backtracking", isto é, o último é removido de S e é adicionado o primeiro vértice viável que é destino de xr-1 na lista de adjacência

P6: o processo termina quando S contém somente o vértice xi e não existe vértice viável que possa ser adicionado a S. Caso contrário voltar para o passo P3

Problema do Caixeiro Viajante

EX.:



$$x_j \notin S$$

S={...}

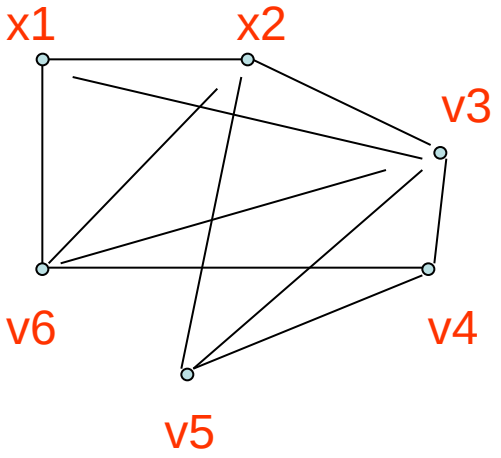
v1	v2	v3	v4	v5	v6
v2	v3 v5	v1 v4	v3 v6	v3 v4	v1 v2 v3

Busca ou Caminhamento em Grafos

- Caminhamento:Processo sistemático de caminhamento pelos vértices e arestas de um grafo
- Algoritmo básico: se G um grafo conexo em que todos os seus vértices estão desmarcados. Marque um vértice arbitrariamente inicialmente. Selecione, agora, algum vértice V que já esteja marcado s seja incidente a alguma aresta (v,w) ainda não explorada. A aresta (v,w) torna-se explorada e o vértice w marcado. O processo termina quando todas as arestas de G tiverem sido exploradas

Busca ou Caminhamento em Grafos

- Algoritmo busca geral
- Escolher e marcar vértice inicial
- Enquanto existir algum vértice V marcado e incidente a uma aresta(v,w) não explorada faça:
  - Escolher o vértice v e explorar a aresta (v,w)
  - Se w é não marcado então marcar w



Busca ou Caminhamento em Grafos

- Busca em Profundidade – DFS(Depth First Search)
  - Uma busca é dita em profundidade quando o critério de escolha de vértice marcado (a partir do qual será realizada a próxima exploração de aresta) obedecer a:
    - “Dentre todos os vértices marcados e incidentes a alguma aresta ainda não explorada, escolher aquele mais recentemente alcançado na busca”

Busca ou Caminhamento em Grafos

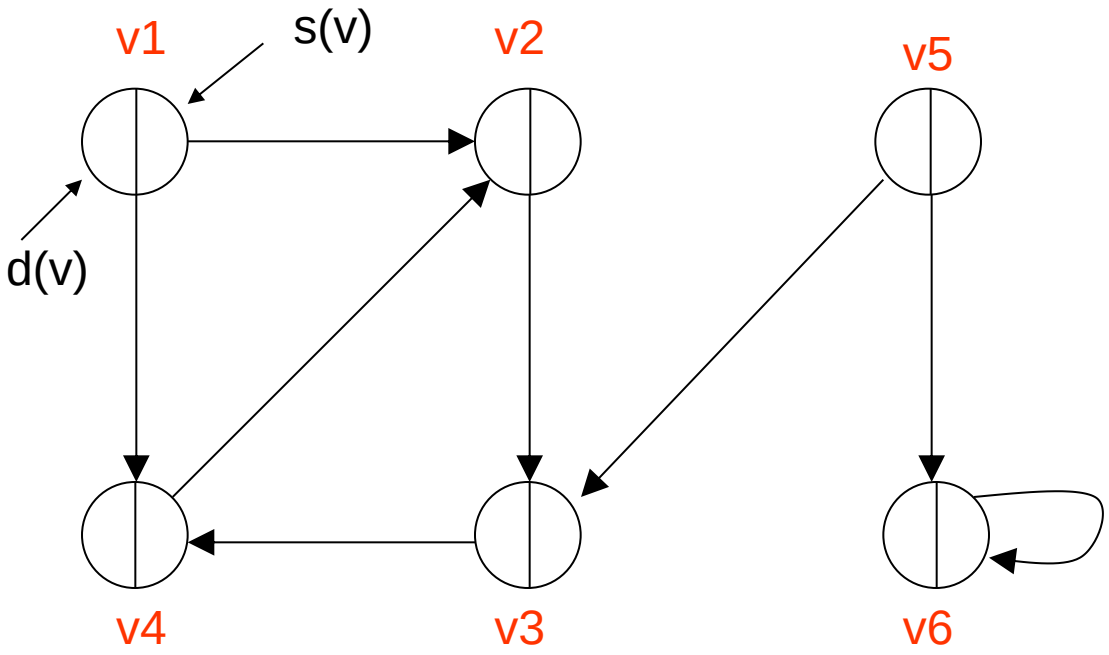
- Algoritmo DFS

```
Para cada vértice v pertencente a V faça
     v.marcar=0;//não visitado
d(v)=0; t=0;
Para cada vértice v pertencente a V faça
     se(v.marcar==0)
        DFS-VISITA(v);

DFS-VISITA(v){
     v.marcar=-1;// visitado
     d(v)=++t;
     Para cada vértice w pertencente a ADJ[v] faça
        se (w.marcar==0)
           DFS-VISITA(w);
     v.marcar=1; // marcado
     s(v)=++t;
}
```

Busca ou Caminhamento em Grafos

- Exemplo DFS



Busca ou Caminhamento em Grafos

- Busca em Largura – BFS(Breadth First Search)
  - Uma busca é dita em largura quando o critério de escolha de vértice marcado (a partir do qual será realizada a próxima exploração de aresta) obedecer a:
    - “Dentre todos os vértices marcados e incidentes a alguma aresta ainda não explorada, escolher aquele menos recentemente alcançado na busca”

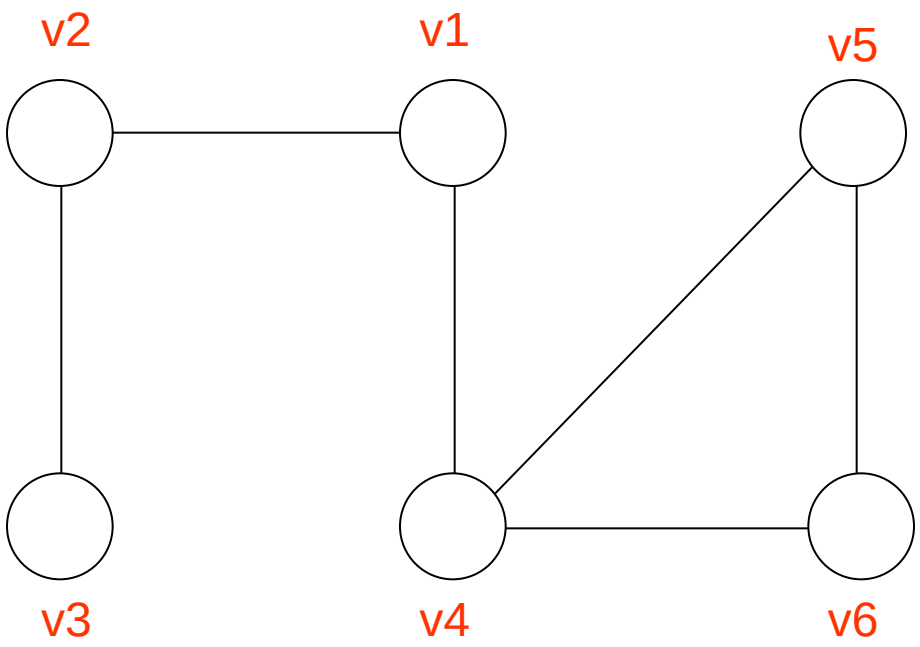
Busca ou Caminhamento em Grafos

- Algoritmo BFS

```
Para cada vértice v pertencente a V -{S} faça
     v.marcar=0;//não visitado
v.marcar=-1;// marcado
d(s)=0;
Q={}
Q.enqueue(S);
Enquanto(Q<>vazio){
     v=Q.dequeue();
     Para cada vértice w adjacente a ADJ[v] faça
        se(w.marcar==0){
          d(w)=d(v)+1;
          w.marcar=-1;
          Q.enqueue(w);
        }
     v.marcar=1; // marcado
}
```

Busca ou Caminhamento em Grafos

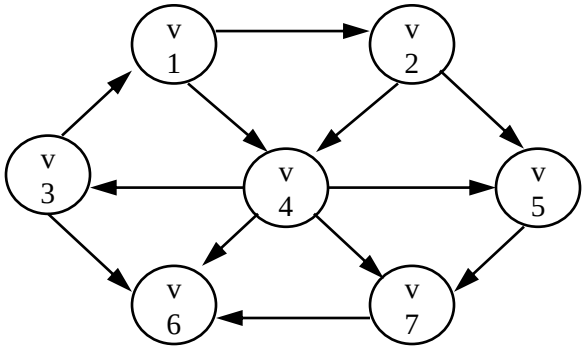
- Exemplo BFS



Q={} v0=

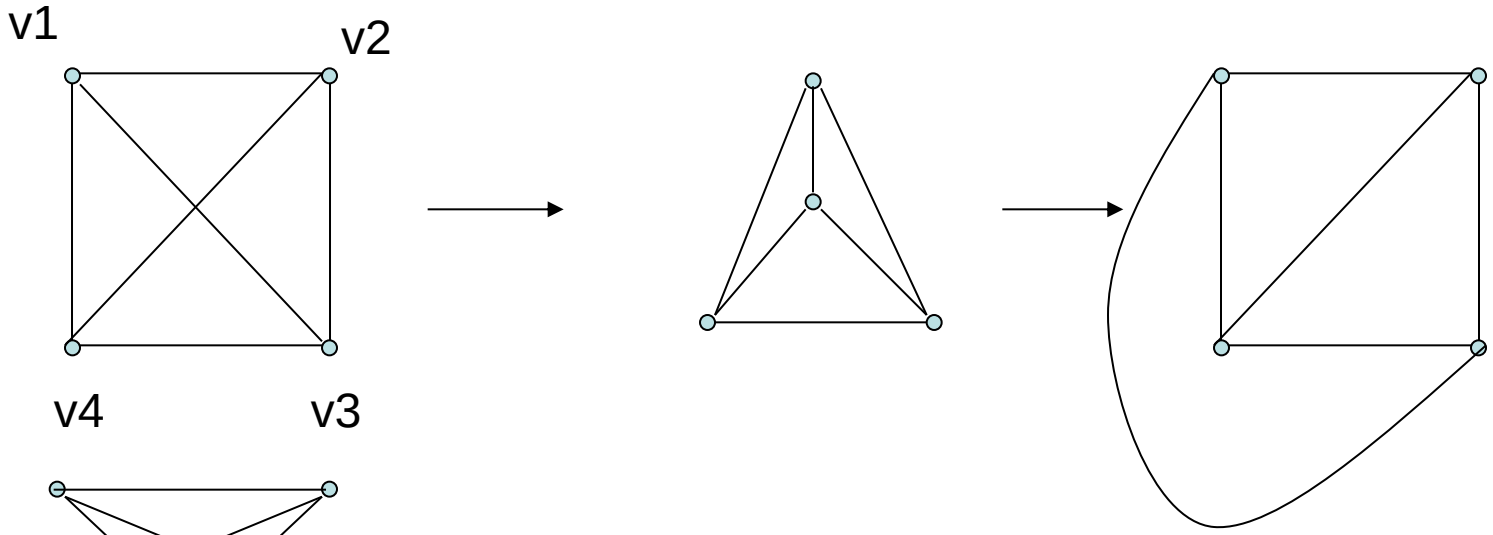
Busca ou Caminhamento em Grafos

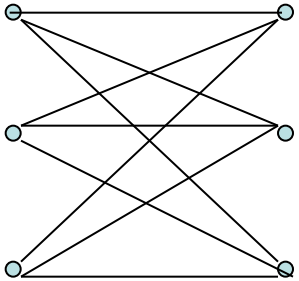
- Exercício DFS e BFS



Grafos Planares

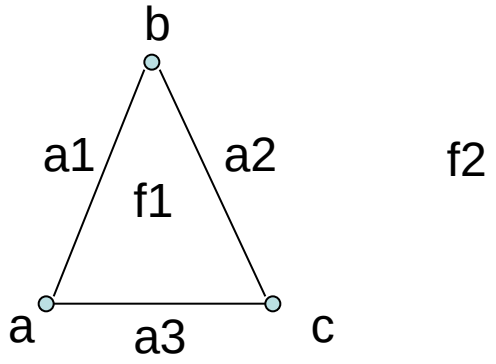
- Um grafo é dito ser planar se existir alguma representação geométrica de G que possa ser desenhada em um plano, de tal modo que não haja cruzamento de arestas





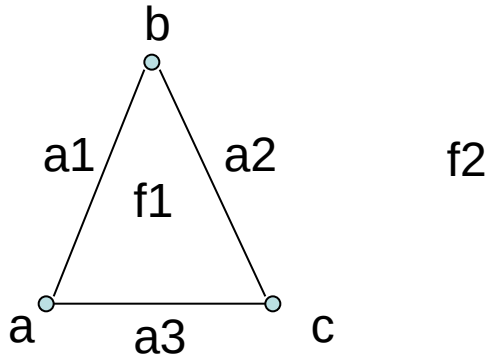
Grafos Planares

- Face: a face de um grafo planar é limitada por arestas do grafo e que não contenha arestas nem vértices no seu interior



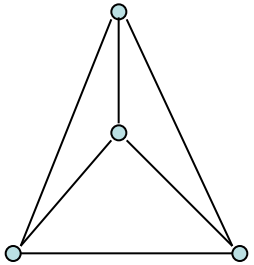
Grafos Planares

- Fronteira: a fronteira de uma face é o conjunto de arestas que a limita
  - `Ex.: a1 a2 a3 -> fronteiras de f1 e f2`



Grafos Planares

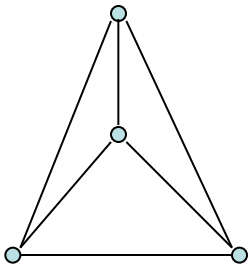
- Primeiro teorema de Euler para o número de faces em um grafo planar
  - Um grafo planar e conexo, com n vértices e m arestas, possui m-n+2=f faces



6 – 4 +2 =4 faces

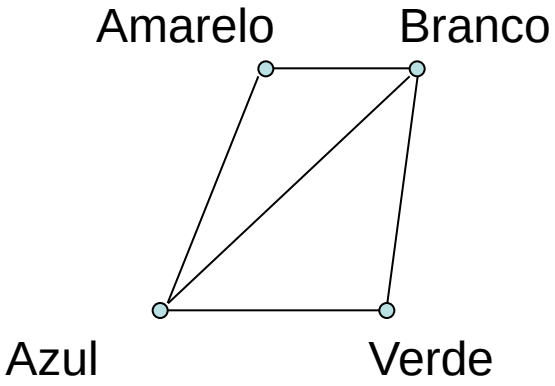
Grafos Planares

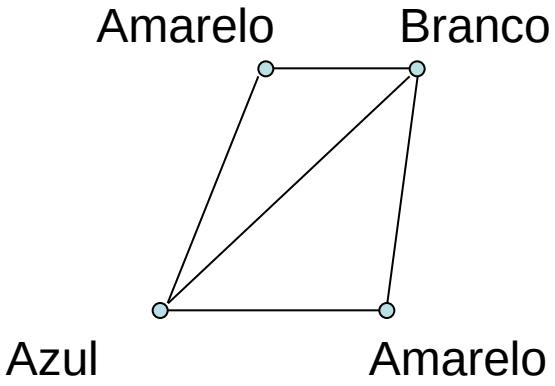
- Teorema 2
  - Todo grafo planar não orientado possui um vértice x com grau <=5



Coloração de Grafos

- Surge quando desejamos colorir os vértices de um grafo de n vértices, de tal modo que nunca dois vértices adjacentes tenham a mesma cor e que se utilize um número de cores mínimo
- O ato de pintar os vértices de um grafo de tal modo que nunca dois vértices adjacentes tenham cores iguais é chamado de coloração do grafo

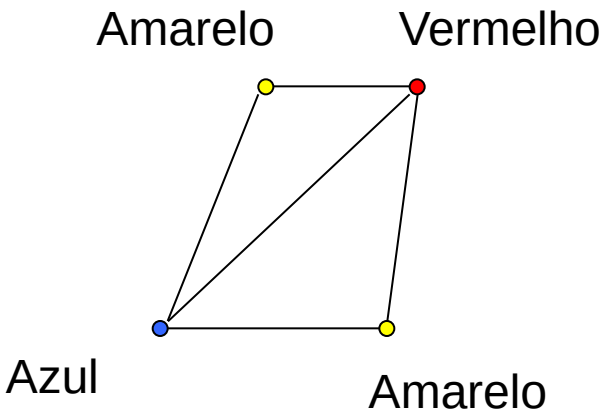




Coloração com número mínimo de cores 3-cromático

Coloração de Grafos

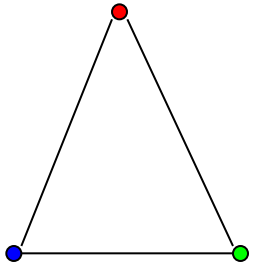
- Número Cromático
  - Um grafo G que exige k-cores para pintar seus vértices, e não menos, é chamado de grupo k-cromático e o número k é chamado de número cromático de G



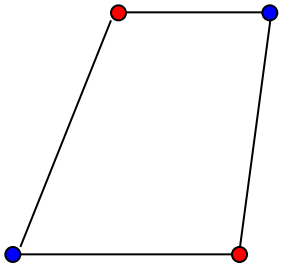
Coloração com número mínimo de cores 3-cromático

Coloração de Grafos

- Observações Derivadas da definição
  - Um grafo constituído de somente vértices isolados é 1-cromático
  - Um grafo com uma ou mais arestas (sem laço) é pelo menos 2-cromático
  - Um grafo completo de n vértices é n-cromático
  - Um grafo consistindo de um ciclo com n>2 vértices é 2-cromático, se n for par e 3-cromático se n for ímpar

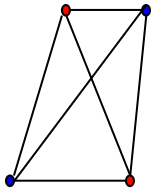


3-cromático



2-cromático

  - Qualquer grafo planar pode ser colorido com no máximo quatro cores
  - Toda árvore com 2 ou mais vértice é 2-cromático
  - Se GRMAX é o grau máximo de um vértice em um grafo G, então o número cromático de G é menor ou igual a GRMAX+1



Nº crom 4 <= 3+1

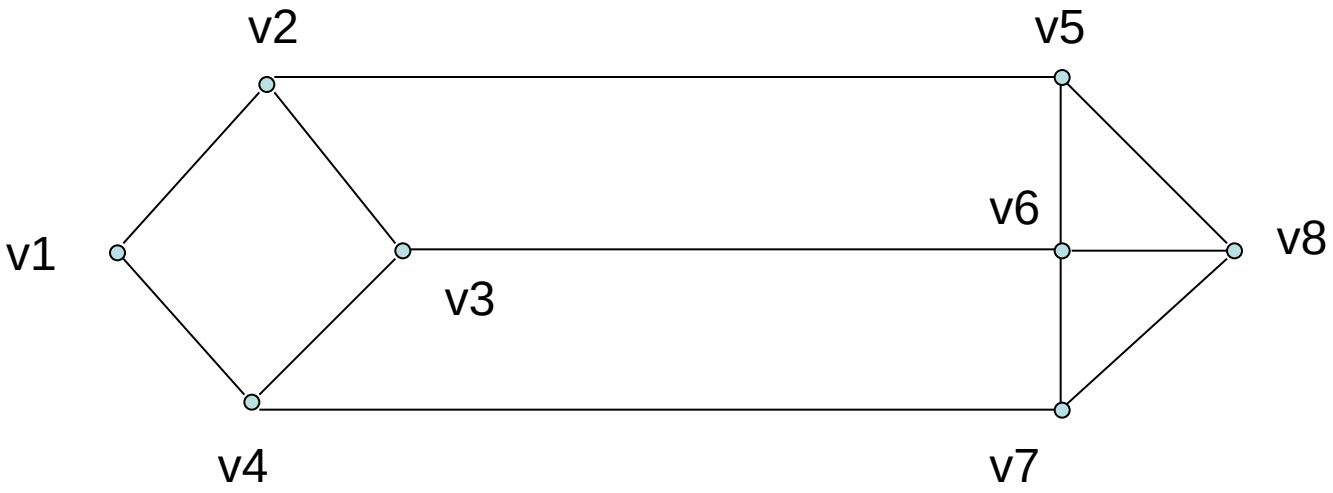
Coloração de Grafos

- Algoritmo para Coloração dos Vértices de um Grafo

```
Entrada: grafo matriz de adjacência)
Saída: Tk, k=1,2,..., onde Tk contém os vértices coloridos com a cor k
P1: sejam v1,v2,...,vk os vértices do grafo G(V,A);
    Coloque os vértices numa lista de tal modo que Gr(vi)>=gr(vj) para todo
   e qualquer vi, vj pertencente a V (ordem crescente de graus)
P2: i=1;
P3: enquanto(v<>vazio){
      coloque o 1° elemento na lista em Ti(vi)
      retire Vj da lista
      enquanto(existir na lista algum vértice vk não adjac.a qualquer vértice de Ti){
                coloque Vk em Ti;
                retire Vk da lista ;
      }
      i++
    }
P4: o número cromático é dado por i-1 e os vértice de mesma cor estão em Ti
```

Coloração de Grafos

- Algoritmo para Coloração dos Vértices de um Grafo



Ti={} i=1

V={v1,v2,v3,v4,v5,v7,v8,v6}